



UNIVERSITÀ
DI SIENA
1240

DIPARTIMENTO DI
INGEGNERIA DELL'INFORMAZIONE E SCIENZE
MATEMATICHE

INGEGNERIA INFORMATICA E DELL'INFORMAZIONE

Curriculum: Sistemi Informatici

REALIZZAZIONE IN ROS DI UN CONTROLLORE INTEGRATO PER LA BALLON HAND

Relatore:

Prof. Gionata Salvietti

Candidato:

Abdourahmane Barro

Mat. 116009

Correlatori:

Dott. Alessio De Bona

Anno Accademico 2024-2025

Indice

1	Introduzione	3
1.1	Struttura Tesi	4
1.2	Stato dell'arte	5
1.2.1	I gripper	5
1.3	Obiettivo tesi	8
2	Architettura	10
2.1	Modello di attuazione ibrido	10
2.1.1	Hardware del Gripper	12
2.1.2	Hardware per il controllo soft	16
2.2	Software	17
2.2.1	ROS	17
3	Implementazione	20
3.1	Nodo <code>PneumaticBox_traj_playback</code>	20
3.1.1	Implementazione Iniziale: L'approccio con <code>goal.msg</code>	20
3.1.2	L'ottimizzazione con <code>goals.msg</code>	22
3.2	Analisi Dettagliata dei Nodi Subscriber	24
3.2.1	Nodo <code>pneumaticbox_traj_handler</code> : Logica e Controllo	24
3.2.2	Misure di Sicurezza Integrate	27

3.2.3	Nodo <code>gripController</code> : Interfaccia di Controllo del Gripper .	31
3.3	CoGripperDongle: Il Ponte Wireless	36
3.3.1	Logica di Ricezione e Parsing Seriale	36
3.3.2	Architettura di Invio Asincrono: Coda e Task Dedicato	38
3.3.3	Protocollo di Comunicazione Wireless: ESP-NOW	40
3.4	Firmware del Gripper: Ricezione ed Esecuzione Comandi	41
3.4.1	Architettura Asincrona con Coda di Comandi (Producer-Consumer)	42
3.4.2	Elaborazione dei Comandi e Controllo Motori	45
3.4.3	Il Ciclo di Controllo Continuo	46
4	Validazione sperimentale	49
4.1	Setup	49
4.2	Validazione del Controllore Pneumatico Bang-Bang	50
4.2.1	Controllo a Setpoint Standard	50
4.2.2	Controllo traiettoria gripper	52
4.2.3	Manipolazione In-Hand: Rotazione di una Sfera	56
5	Conclusioni e Sviluppi Futuri	58
5.1	Conclusioni	58
5.2	Sviluppi Futuri: Reinforcement Learning per la Manipolazione Auto-	
	noma	59
	Appendice A	61
	Bibliografia	62

Capitolo 1

Introduzione

La robotica è lo studio e la pratica interdisciplinare della progettazione, costruzione, funzionamento e utilizzo dei robot [1]. Questa definizione basilare è stata profondamente ampliata da decenni di progressi tecnologici. La robotica moderna ha superato i confini della semplice automazione, evolvendosi in un campo altamente interdisciplinare che si concentra sulla creazione di sistemi intelligenti, adattivi e spesso collaborativi. Il paradigma contemporaneo è caratterizzato da un significativo spostamento del focus dai robot che operano in ambienti industriali altamente strutturati e prevedibili, a contesti più complessi, dinamici e non strutturati. In questo scenario, l'importanza di gripper avanzati è cruciale. Per esempio, nella robotica assistiva, un gripper deve essere in grado di manipolare oggetti con delicatezza, variabilità e precisione, adattandosi a forme e consistenze diverse - dal raccogliere un bicchiere d'acqua al maneggiare un oggetto fragile. Questo richiede una sofisticata integrazione tra sensori, modellazione e materiali.

1.1 Struttura Tesi

Il presente lavoro di tesi è organizzato come segue per illustrare in modo chiaro e sequenziale il percorso di sviluppo.

Capitolo 1: verrà introdotto il contesto del progetto, partendo dallo stato dell'arte dei gripper robotici e analizzando i risultati dei lavori precedenti. Si definiranno gli obiettivi specifici di questa tesi, incentrati sull'integrazione dei sistemi di attuazione e sull'implementazione di un controllo unificato.

Capitolo 2: si entrerà nel dettaglio del design hardware e dell'architettura software. Verranno descritti i componenti del sistema, la struttura dei nodi ROS, i topic utilizzati per la comunicazione e la logica di controllo implementata per la gestione sinergica del gripper.

Capitolo 3: sarà dedicato al processo di implementazione. Verranno discusse le sfide tecniche affrontate durante lo sviluppo, come la stabilità del controllo pneumatico e l'integrazione dei diversi moduli software, e saranno presentate le soluzioni adottate per superarle.

Capitolo 4: verrà illustrata la metodologia sperimentale utilizzata per validare il sistema. Saranno presentati e discussi i risultati ottenuti dai test di presa e manipolazione su diversi oggetti, analizzando le performance del sistema integrato.

Capitolo 5: concluderà il lavoro riassumendo i contributi principali della tesi. Verranno inoltre discusse le prospettive future, delineando possibili miglioramenti e nuove direzioni di ricerca per il progetto.

1.2 Stato dell'arte

1.2.1 I gripper

Se il braccio robotico offre al robot le capacità di estensione e posizionamento, è il gripper, o end-effector, la componente che gli permette di interagire con il mondo [2].

Un gripper robotico è uno strumento EOAT (end of arm tooling) che si aggancia al polso di un braccio robotico progettato specificamente per interagire con l'ambiente afferrando, mantenendo e manipolando oggetti.

Funziona come l'interfaccia fisica tra la computazione del robot e il mondo fisico. È, in sostanza, la mano del robot, e il suo design e le sue capacità sono fondamentali per la buona riuscita di qualsiasi compito di manipolazione [3]. La scelta del gripper è dettata in modo decisivo dall'applicazione prevista e, soprattutto, dalle caratteristiche fisiche degli oggetti da maneggiare: la loro dimensione, forma, peso e fragilità. Il ruolo del gripper è quello di stabilire e mantenere un contatto sicuro con un oggetto, assicurando che la sua posizione e il suo orientamento siano controllati durante l'operazione, che si tratti di manipolare tessuti biologici in un'operazione chirurgica o semplicemente raccogliere un frutto [4].

Per ottenere una presa stabile e affidabile, i gripper robotici impiegano principalmente uno di questi due principi fondamentali: la presa di forza (force-lock) o la presa di forma (form-lock) [5]. La scelta tra queste strategie costituisce una decisione progettuale fondamentale che determina la complessità del meccanismo, i requisiti di controllo e l'efficacia per applicazioni specifiche.

Nella presa di forma, la stabilità si ottiene vincolando geometricamente l'oggetto, "ingabbiandolo" o avvolgendolo efficacemente tra le dita e il palmo del gripper. La restrizione è generata quasi esclusivamente dalla forma della presa stessa, piuttosto che da elevate forze di compressione. Ciò permette alla presa di essere stabile con una forza applicata minima, che viene poi distribuita su un'area di contatto molto

più ampia [5]. Questo principio è formalmente noto come *form closure*, una condizione più robotusta alla *force closure*, in cui l'oggetto è vincolato anche con contatti senza attrito [6].

Nella presa di forza, invece, l'oggetto viene immobilizzato unicamente attraverso l'applicazione di forze tra le superfici di contatto del gripper e l'oggetto stesso [5]. Queste forze, tipicamente normali e di attrito, devono essere continuamente e precisamente controllate per contrastare forze esterne come la gravità e l'inerzia, impedendo all'oggetto di scivolare. Una presa di forza di successo richiede che le "coppie di contatto" possano resistere a qualsiasi coppia esterna arbitraria; questa condizione è formalmente nota come *force closure*. Questo metodo è comune nei semplici gripper a ganasce che si basano sul "pizzicare" un oggetto [6].

La mano umana è l'ispirazione e il punto di riferimento ultimo, dotata della notevole destrezza di passare senza soluzione di continuità tra prese con la punta delle dita (una forma di presa di forza) e potenti prese avvolgenti (presa di forma), a seconda del compito [5]. Questi due principi non sono concetti astratti; rappresentano una fondamentale dicotomia di design che si traduce direttamente nel più ampio paradigma della robotica "rigida contro soffice".

Convenzionalmente, i gripper rigidi, spesso caratterizzati da semplici ganasce non articolate che creano contatti puntuali o lineari, sono intrinsecamente orientati verso una strategia di presa di forza. [7] La loro incapacità di conformarsi alla geometria di un oggetto significa che devono fare affidamento sull'applicazione di una sufficiente "forza di stretta" per generare l'attrito necessario a una presa stabile. Questo è un classico approccio basato sulla presa di forza. In netto contrasto, i soft gripper, per loro stessa definizione, sono costruiti con materiali conformi progettati per deformarsi e adattarsi alle superfici degli oggetti. Questa deformazione passiva crea naturalmente una presa avvolgente o conforme, che è la rappresentazione fisica del principio della presa di forma. Questa distinzione porta a una cruciale implica-

zione di secondo ordine per la progettazione del sistema. Poiché i gripper rigidi si basano su forze elevate e concentrate, necessitano di sensori di forza sofisticati e spesso costosi, e di complessi algoritmi di controllo a feedback per modulare la loro stretta, prevenendo danni all'oggetto o scivolamenti [8]. I soft gripper, invece, ottengono prese stabili e intrinsecamente delicate grazie alle proprietà materiali e alla loro morfologia - un concetto noto come intelligenza incorporata (embodied intelligence). Questo scarica una parte significativa della complessità del controllo dal software all'hardware fisico stesso. Di conseguenza, la principale sfida di controllo per i soft gripper si sposta spesso dalla precisa regolazione della forza al controllo della forma e della posizione complessiva. La scelta tra un gripper rigido e uno soffice è, quindi, essenzialmente una scelta tra il privilegiare una strategia di presa di forza con le sue associate sfide di controllo o una presa di forma con la sua intrinseca sicurezza ed adattabilità.

Per superare i semplici movimenti pre-programmati e ottenere una presa intelligente e adattiva, l'integrazione di sensori è essenziale. Questi sensori forniscono il feedback necessario per un sistema di controllo a circuito chiuso, trasformando la pinza da un mero strumento meccanico in un sistema mecatronico intelligente. Le informazioni sensoriali utilizzate nella presa possono essere ampiamente suddivise in due tipi:

1. **Percezione Esteroceettiva:** Questa implica la raccolta di informazioni sul mondo esterno, inclusi l'oggetto da afferrare e la sua relazione con la pinza [5].

Questa categoria include:

- **Sensori di Prossimità:** Rilevano la presenza di un oggetto entro un certo raggio senza contatto fisico, segnalando alla pinza di prepararsi per la presa.
- **Sistemi di Visione:** Telecamere e altri sensori di imaging (es. LiDAR, sensori di profondità) vengono utilizzati per identificare, localizzare e

determinare l'orientamento di un oggetto, fornendo dati cruciali per la pianificazione della presa.

- **Sensori Tattili:** Questi sensori forniscono informazioni sul contatto fisico stesso, come la distribuzione della pressione sulla superficie di contatto. Possono essere semplici interruttori binari che confermano il contatto o complessi array di informazioni che forniscono un ricco "senso del tatto".

2. **Percezione Propriocettiva:** questa implica il monitoraggio dello stato interno della pinza stessa: la sua configurazione e le forze che sta esercitando.

Questa categoria include:

- **Sensori di Posizione/Velocità:** Dispositivi come encoder e potenziometri misurano la posizione e il movimento delle giunture e delle dita della pinza, permettendo al controllore di conoscere la configurazione della pinza in ogni momento.
- **Sensori di Forza/Coppia:** Questi sensori sono integrati nella struttura della pinza (ad esempio, nelle giunture o nelle punte delle dita) per misurare l'entità delle forze di presa applicate. Questo feedback è fondamentale per ottenere una presa stabile senza schiacciare l'oggetto [9].

1.3 Obiettivo tesi

Il presente lavoro di tesi descrive l'evoluzione del SoftPad Gripper [5], qui sviluppato nel progetto 'Ballon Hand', e si concentra sull'implementazione di un sistema di controllo integrato. In continuità con le ricerche precedenti svolte presso il laboratorio, l'obiettivo è realizzare la coesistenza e la cooperazione tra il sistema di attuazione a tendini, responsabile del movimento di chiusura delle dita del gripper, e quello pneumatico, che controlla i polpastrelli soft per l'adattamento al contatto. Tale in-

tegrazione è finalizzata a ottimizzare l'efficacia della presa e a implementare routine di manipolazione dell'oggetto, superando i limiti dei sistemi di controllo disgiunti.

Capitolo 2

Architettura

Questo capitolo fornisce un'analisi dettagliata dell'architettura del gripper, esaminando sistematicamente i suoi sottosistemi costituenti. La prima parte del capitolo è dedicata all'implementazione fisica del sistema: la filosofia di progettazione generale, l'intricata struttura meccanica e il sistema di attuazione primario a tendini e della parte di attuazione pneumatica. La seconda parte tratterà il framework software unificante (ROS). Nell'appendice A saranno presenti dettagli aggiuntivi sull'hardware, mentre all'interno della repository [11] saranno presenti per intero i codici usati.

2.1 Modello di attuazione ibrido

L'architettura del gripper si fonda su una filosofia di progettazione ibrida, che fonde strategicamente due distinti paradigmi di attuazione per creare un sistema più capace della somma delle sue parti. Questo approccio affronta direttamente le ben documentate limitazioni dei gripper puramente rigidi o puramente morbidi (come già discusso nel capitolo 1). Mentre i gripper rigidi tradizionali offrono elevata forza e precisione posizionale, incontrano difficoltà in ambienti non strutturati e con oggetti

di forma irregolare. Al contrario, i gripper morbidi forniscono eccellente cedevolezza (compliance) e adattabilità, ma spesso mancano della forza, della precisione e della durabilità richieste per molte applicazioni industriali. L'obiettivo fondamentale di questo lavoro è ottenere una combinazione sinergica di destrezza, adattabilità e applicazione controllata della forza, integrando un robusto sistema sottoattuato a tendini per la presa primaria con un sistema pneumatico cedevole per il contatto adattivo.

Il concetto ibrido al centro del progetto combina una struttura scheletrica rigida, stampata in 3D, con superfici di contatto cedevoli, attuate pneumaticamente e integrate nelle dita. Il sistema a tendini fornisce l'integrità strutturale e la forza di presa primaria, in modo analogo a un sistema muscolo-scheletrico biologico [3]. È perciò responsabile del movimento macroscopico delle dita, chiudendole attorno a un oggetto. I pad gonfiabili, che verranno ulteriormente discussi successivamente, forniscono cedevolezza e un tocco delicato. Possono alterare dinamicamente il loro volume per conformarsi alla geometria locale di un oggetto, aumentando significativamente l'area di contatto e garantendo una presa stabile e sicura. Questo approccio ibrido, che combina elementi rigidi e morbidi, è una tendenza di primo piano nella ricerca robotica avanzata, volta a creare end effectors finali più versatili e capaci.

Tuttavia, per implementare efficacemente questi sistemi ibridi, è necessario discutere del concetto di meccanismo sottoattuato, che per definizione, possiede meno attuatori che gradi di libertà, il che ne semplifica il controllo, riduce il peso e abbassa i costi. Tuttavia, questa semplificazione avviene a scapito della controllabilità; è impossibile comandare una traiettoria arbitraria per ogni falange in modo indipendente. Di conseguenza, durante la presa di un oggetto, in particolare uno con una geometria non banale, un dito sottoattuato si conformerà passivamente. Ciò si traduce spesso in un contatto limitato a pochi punti di alta pressione o agli spigoli vivi delle falangi, piuttosto che a un contatto superficiale distribuito e stabile.

Questo fenomeno è stato osservato sperimentalmente, dove una presa iniziale che utilizzava solo i tendini è stata descritta come "debole e transitoria" [3]. I pad gonfiabili non sono quindi semplicemente una caratteristica ausiliaria per la "morbidezza", ma un meccanismo compensativo diretto ed essenziale per la limitazione intrinseca del metodo di attuazione primario. Gonfiandosi, i pad si espandono per colmare il divario geometrico tra le falangi rigide e la superficie dell'oggetto. Questa azione trasforma contatti puntiformi instabili in contatti superficiali stabili e di ampia area, migliorando drasticamente la qualità e la robustezza della presa. Ciò rivela una logica di progettazione profondamente integrata: il sistema pneumatico è una soluzione a un problema creato dalla scelta della sottoattuazione, rendendo l'intero sistema sinergico.

2.1.1 Hardware del Gripper

Attuatori: Servomotori Dynamixel MX-28

I motori scelti per l'azionamento a tendine sono tre servomotori ROBOTIS Dynamixel MX-28. In quanto questi componenti sono attuatori "all-in-one" le cui caratteristiche integrate sono cruciali per l'architettura di controllo. La decisione è stata guidata da un equilibrio di specifiche appropriate per il compito: una coppia di stallo di $2.5 N \cdot m$ (a 12V) fornisce potenza sufficiente per un'ampia gamma di applicazioni di presa, mentre un'elevata risoluzione di posizione di 4096 passi per giro (equivalente a 0.088°) consente un controllo preciso sulla flessione delle dita [3]. Oltre alle pure prestazioni, gli attuatori Dynamixel offrono due vantaggi a livello di sistema che hanno influenzato profondamente l'architettura del gripper. Innanzitutto, dispongono di un protocollo di comunicazione seriale TTL half-duplex che supporta una topologia daisy-chain (vedi fig. 2.1). Questa è una scelta hardware critica che semplifica drasticamente il cablaggio del sistema. Invece di una tradizio-

nale topologia a stella (vedi fig. 2.2), in cui ogni motore richiede una connessione separata e dedicata al controllore centrale, un singolo bus a 3 fili (alimentazione, massa e dati) può essere instradato da un motore all'altro, controllando tutti gli attuatori in serie. Questa architettura riduce significativamente peso, volume, complessità e il numero di potenziali punti di guasto, il che è una strategia comune ed efficace nei sistemi robotici multi-attuatore.

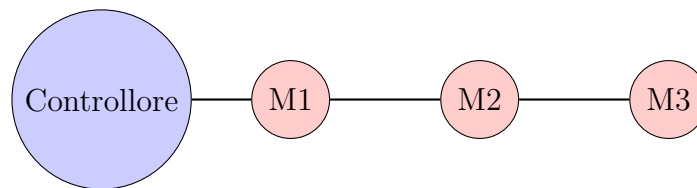


Figura 2.1: Topologia daisy-chain: i motori sono collegati in serie, riducendo il cablaggio.

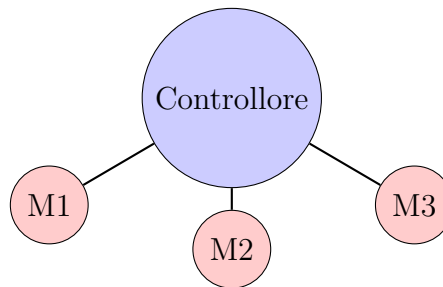


Figura 2.2: Topologia a stella: ogni motore è connesso direttamente al controllore.

Ogni Dynamixel contiene un microcontrollore di bordo e una mappa di memoria, la *Control Table*, che consente a un controller host di leggere dati dai sensori e scrivere parametri di controllo direttamente nei registri del motore tramite comunicazione a pacchetti [3]. Questo progetto sfrutta ampiamente questa funzionalità per delegare compiti di basso livello dal microcontrollore principale. Parametri come la Modalità Operativa (OP_EXTENDED_POSITION), la Velocità di Profilo, l'Accelerazione di Profilo e i Limiti di PWM/Corrente vengono configurati direttamente sui motori stessi. Il registro fondamentale è quello posto all'indirizzo 126, Present Load, un feedback che fornisce un valore proporzionale alla corrente assorbita dal motore,



Figura 2.3: Dynamixel MX-28

che funge da efficace proxy per il carico meccanico esercitato, questo permette di implementare un rilevatore della forza senza sensori [3].

Interfaccia di Controllo

Il sistema di attuazione a tendini è gestito da una scheda WEMOS D1 mini, integrata nel sistema tramite una scheda a circuito stampato (PCB) progettata su misura [3]. Questa PCB è un componente hardware critico che risolve sfide chiave di alimentazione e comunicazione. Il microcontrollore WEMOS D1 mini funge da controller di alto livello, responsabile dell'esecuzione del ciclo di controllo principale e dell'invio di comandi ai motori Dynamixel. La PCB custom, progettata da SIRSLab, agisce come hub di integrazione centrale per il sottosistema elettromeccanico.

- **Regolazione dell’Alimentazione:** la scheda è alimentata a 12V, tensione compatibile con i motori Dynamixel. Integra un convertitore step-down che fornisce 3.3V alla WEMOS D1 mini, permettendo a entrambi i componenti di funzionare da un’unica fonte.

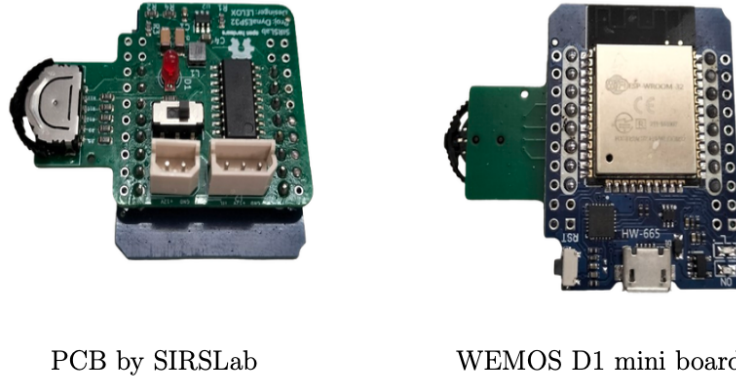


Figura 2.4: [3]

- **Traslazione del Livello Logico (Logic Level Shifting):** una sfida importante è la differenza nei livelli logici: la WEMOS lavora a 3.3V, mentre il bus Dynamixel usa logica TTL a 5V. La PCB custom integra circuiti di traslazione bidirezionali per adattare i segnali tra i due dispositivi, proteggendo il microcontrollore e garantendo una comunicazione stabile.
- **Gestione della Comunicazione Half-Duplex:** il protocollo Dynamixel utilizza una singola linea dati sia per trasmettere (Tx) che ricevere (Rx). Poiché la WEMOS ha pin Tx e Rx separati, la PCB include un circuito di commutazione (basato sul buffer 74HC126, controllato da un pin di direzione della WEMOS) che collega dinamicamente la linea dati del bus al pin corretto nel momento giusto. In questo modo, il microcontrollore può comunicare correttamente sul bus half-duplex.

Per abilitare la comunicazione tra il sistema di controllo del gripper e la macchina host che esegue ROS, è stato impiegato un modulo LILYGO T-Dongle S3. Questo dispositivo compatto, dal formato di una chiavetta USB, è basato sul microcontrollore dual-core ESP32-S3 di Espressif.

La scelta di questo componente è strategica per le sue funzionalità integrate. Il modulo offre connettività wireless nativa Wi-Fi 802.11 b/g/n e Bluetooth 5 (BLE), consentendo uno scambio dati flessibile e senza fili con la macchina host. Il processore dual-core Xtensa LX7 fornisce la potenza di calcolo necessaria per gestire contemporaneamente i protocolli di comunicazione e la logica applicativa richiesta per l'interfacciamento con il gripper, fungendo da ponte efficiente tra i due sistemi.

2.1.2 Hardware per il controllo soft

Pneumaticbox

Questo strumento è progettato per gestire con precisione il flusso d'aria attraverso sei canali indipendenti, utilizzando elettrovalvole temporizzate caricate da un compressore esterno. L'operatività del sistema si basa su un controllo discreto: le elettrovalvole, normalmente chiuse, vengono aperte per una durata specifica definita dall'utente tramite delle API fornite dal costruttore. L'utente può selezionare un canale specifico e la direzione del flusso, ovvero verso l'esterno per l'infrazione o verso l'interno per la deflazione.

Una caratteristica hardware fondamentale è l'integrazione di sensori di pressione relativa su ciascuno dei sei canali, che forniscono un feedback costante sul livello di gonfiaggio di ogni pad.

Tuttavia, questa architettura presenta delle criticità intrinseche. La natura discreta del controllo, basata unicamente sul tempo di apertura delle valvole, può risultare limitante in scenari che richiedono una regolazione più fine e continua del flusso d'aria. Inoltre, il sistema dipende esclusivamente dalla pressione erogata dal compressore; un inevitabile calo di pressione nel tempo può compromettere l'accuratezza e la ripetibilità del controllo. Questo calo può essere causato dall'apertura continua delle valvole durante routine che prevedono molteplici inflazioni in tempi ristretti,

portando il compressore a operare sotto la soglia ottimale di 300-400 kPa, o da perdite d'aria dai pad pneumatici stessi. Tale situazione richiede un monitoraggio costante del sistema.

2.2 Software

2.2.1 ROS

Introduzione a ROS

Un componente fondamentale dell'architettura della pinza è la sua dipendenza dal Robot Operating System (ROS) come framework software unificante. ROS non è un sistema operativo nel senso tradizionale, ma piuttosto un framework flessibile per la scrittura di software robotico. Fornisce una collezione di strumenti, librerie e convenzioni che semplificano il compito di creare applicazioni robotiche complesse.

I concetti chiave all'interno di ROS includono:

- **Nodi:** Si tratta di processi eseguibili che svolgono calcoli. Ogni nodo è progettato per eseguire un compito specifico, come controllare un motore, leggere dati da sensori o pianificare percorsi.
- **Topic:** I nodi comunicano tra loro inviando e ricevendo messaggi tramite i topic. I topic sono canali nominati su cui i nodi possono pubblicare messaggi o iscriversi per riceverli. Questo modello publisher/subscriber consente una comunicazione a basso accoppiamento tra le diverse parti del sistema robotico, favorendo uno sviluppo software modulare e distribuito.

Architettura ROS del sistema

Il sistema di controllo del gripper si basa su un'architettura ROS progettata per orchestrare in modo sincrono i suoi componenti eterogenei: un sistema pneumatico per l'attuazione soft e un servomotore per la presa. Come illustrato in Figura 2.5, questa architettura è composta da tre nodi principali che comunicano attraverso un singolo topic, garantendo che le azioni della parte pneumatica e del gripper meccanico siano coordinate.

Nodi del Sistema

- *pneumaticbox_traj_playback*: Questo nodo funge da direttore del sistema. Il suo compito è leggere una traiettoria predefinita da un file di testo. Ogni riga del file specifica un setpoint di sei pressioni (una per ogni canale pneumatico) e un comando per il gripper (ad esempio, 'apri', 'chiudi'). Il nodo pubblica queste informazioni in sequenza sul topic di comunicazione, dettando il comportamento desiderato dell'intero sistema nel tempo.
- *pneumaticbox_traj_handler*: Il primo nodo "attuatore", scritto in Python, si iscrive al topic di comunicazione per ricevere gli obiettivi di pressione. Una volta ricevuto un messaggio, questo nodo interagisce direttamente con il controller della **Pneumaticbox**. Implementa una logica di controllo (nello specifico, un controllore bang-bang) per gonfiare o sgonfiare attivamente ciascun canale pneumatico fino al raggiungimento della pressione target specificata dal nodo *traj_playback*.
- *gripController*: Il secondo nodo "attuatore", anch'esso in Python, opera in parallelo al *traj_handler*. Si iscrive allo stesso topic, ma ignora i dati di pressione e si concentra esclusivamente sul campo 'grip' del messaggio. Quando riceve un nuovo comando, lo inoltra tramite comunicazione seriale all'hardware

del gripper (un servomotore Dynamixel), eseguendo così le azioni di presa e rilascio.

Topic di Comunicazione *pressure_goal*: È il canale di comunicazione centrale dell'architettura. Tutti i nodi sono connessi a questo topic. Il nodo '*traj_playback*' pubblica messaggi di tipo '*goals.msg*', che contengono due informazioni fondamentali: un array con i valori di pressione desiderato (*pressure_goal*) e una stringa di comando per il gripper ('*grip*'). Questa struttura del messaggio permette di inviare un comando completo e sincronizzato a entrambi i sistemi di attuazione con una singola pubblicazione.

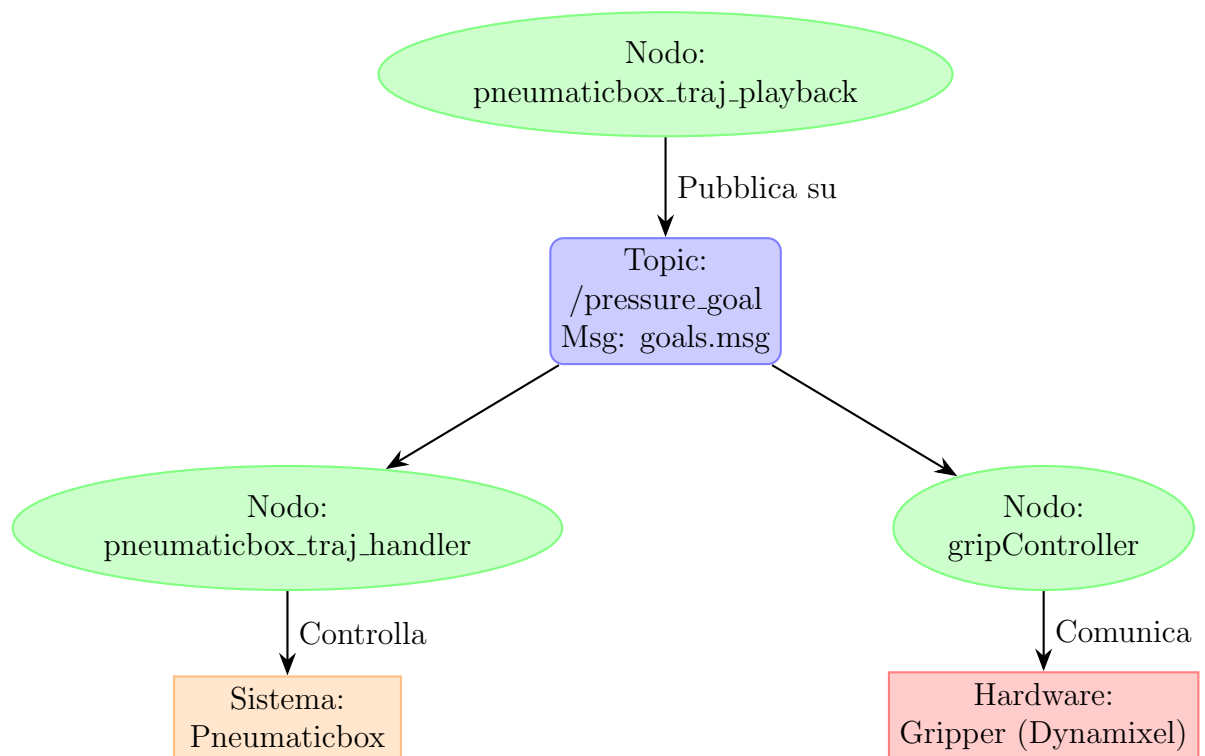


Figura 2.5: Flusso di messaggi e controllo tra i nodi ROS e gli attuatori fisici

Capitolo 3

Implementazione

3.1 Nodo `PneumaticBox_traj_playback`

Il nodo `pneumaticbox_traj_playback` è un componente centrale nell'architettura di controllo, incaricato di leggere una traiettoria predefinita da un file di testo e di pubblicarla su un topic ROS. Questa traiettoria determina il comportamento sincronizzato dei pad pneumatici soft e del gripper meccanico. Un'analisi della sua implementazione iniziale ha rivelato una significativa inefficienza nel suo protocollo di comunicazione, che è stata successivamente risolta attraverso una riprogettazione della struttura del messaggio ROS.

3.1.1 Implementazione Iniziale: L'approccio con `goal.msg`

La prima versione del sistema utilizzava un tipo di messaggio personalizzato chiamato `goal.msg`. La sua struttura è definita come segue:

```
1 int32 id
2 float32 pressure_goal
3 string grip
```

Listing 3.1: Definizione di `goal.msg`

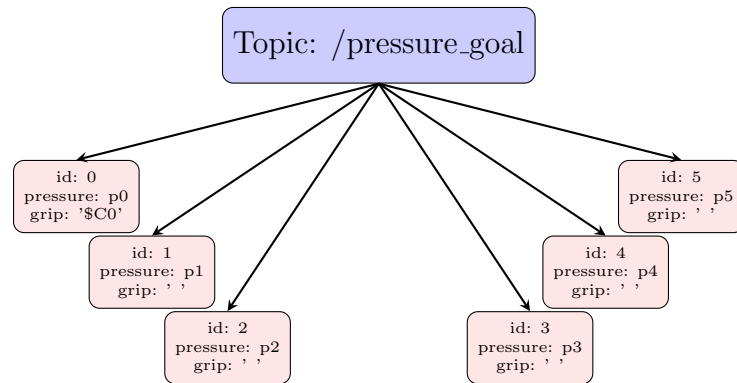
L'intento progettuale era che il campo `id` specificasse quale dei sei canali pneumatici fosse l'obiettivo, mentre `pressure_goal` impostava la pressione desiderata per quel canale. Il campo `grip` conteneva il comando per il gripper a tendini.

La logica all'interno del nodo publisher, `pneumaticbox_traj_playback.cpp`, itera attraverso i sei valori di pressione per un singolo passo temporale e pubblica un messaggio distinto per ciascuno di essi.

```
1 // pneumaticbox_traj_playback.cpp
2 for(size_t i = 0; i < 6; ++i){ // Itera attraverso i 6 canali di
   pressione
3     pneumaticbox_traj_playback::goal msg;
4     msg.id = i; // ID del canale (da 0 a 5)
5     msg.pressure_goal = current_pressure_set[i];
6     if(i==0)
7         msg.grip = current_grip;
8     pub.publish(msg);
9 }
```

Listing 3.2: Loop di pubblicazione originale in `pneumaticbox_traj_playback.cpp`

Questa implementazione, sebbene funzionante, introduce un considerevole overhead. Per ogni passo temporale della traiettoria, vengono pubblicati 6 messaggi separati sul topic `/pressure_goal`. Un'inefficienza critica deriva dalla gestione del campo `grip`. Il comando viene popolato solo nel primo messaggio della sequenza (quando `i==0`), lasciandolo vuoto per i successivi cinque messaggi. Ciò si traduce in un traffico di rete ridondante e richiede che i nodi subscriber contengano una logica per filtrare e agire solo sulle parti rilevanti del flusso di messaggi. Il `pneumaticbox_traj_handler` deve ispezionare l'`id` per aggiornare il canale corretto, mentre il `gripController` deve processare tutti e 6 i messaggi solo per agire sul primo e ignorare gli altri.



Sei messaggi separati per ogni passo temporale

Figura 3.1: Flusso di messaggi inefficiente utilizzando `goal.msg`.

3.1.2 L'ottimizzazione con `goals.msg`

Per affrontare i limiti del design iniziale, la struttura del messaggio è stata riprogettata. Il nuovo messaggio, `goals.msg`, consolida tutte le informazioni necessarie in un'unica unità coerente.

```
1 float32[6] pressure_goal
2 string grip
```

Listing 3.3: Definizione del messaggio ottimizzato `goals.msg`

Questa struttura rivista utilizza un array di float a dimensione fissa per contenere simultaneamente tutti e 6 i setpoint di pressione. Il campo `id` è reso obsoleto, poiché l'indice dell'array corrisponde implicitamente all'ID del canale pneumatico. La logica di pubblicazione nel nodo, è di conseguenza semplificata, eliminando la necessità di un loop per inviare i dati dei singoli canali.

```
1 // pneumaticbox_traj_playback.cpp
2 pneumaticbox_traj_playback::goals msg;
3 for(size_t i = 0; i < 6; ++i){ // Popola l'array
4     msg.pressure_goal[i]=pressures[currentIndex_][i];
5 }
6 msg.grip=grip_commands_[currentIndex_];
```

```

7 pub_.publish(msg);
8 ROS_INFO("Published command");

```

Listing 3.4: Logica di pubblicazione ottimizzata in `pneumaticbox_traj_playback.cpp`

Questo cambiamento architetturale produce benefici significativi:

- **Traffico di Rete Ridotto:** Il numero di messaggi richiesti per trasmettere lo stato di un punto della traiettoria è ridotto da 6 a 1. Ciò porta a una diminuzione di 6 volte dell'overhead dell'header del messaggio e riduce sostanzialmente il consumo di banda sulla rete ROS.
- **Trasferimento Atomico dei Dati:** L'intero comando (tutte e 6 le pressioni e l'azione del gripper) viene ora inviato come un'unica unità atomica. Ciò garantisce che i nodi subscriber ricevano un setpoint completo e sincronizzato, eliminando potenziali race condition o inconsistenze di stato che potrebbero derivare dall'elaborazione di un flusso di messaggi frammentato.
- **Logica dei Subscriber Semplificata:** Sia il node `pneumaticbox_traj_handler` che il `gripController` sono semplificati. Ora ricevono un singolo messaggio contenente tutte le informazioni di cui hanno bisogno per eseguire le rispettive azioni per un dato passo temporale. Non è più necessario ignorare i campi `grip` vuoti o ricostruire lo stato da più messaggi.

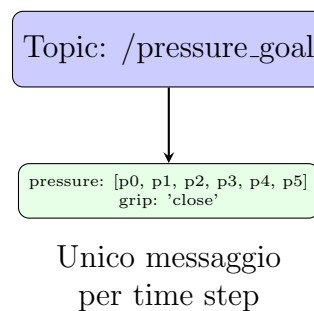


Figura 3.2: Flusso di messaggi efficiente utilizzando il messaggio consolidato `goals.msg`.

In conclusione, l'evoluzione da `goal.msg` a `goals.msg` rappresenta un'ottimizzazione critica dell'architettura di comunicazione del sistema. Passando da un approccio frammentato e multi-messaggio a una struttura consolidata a messaggio singolo, il sistema guadagna in efficienza, robustezza e semplicità, aderendo più strettamente alle best practice nella progettazione di sistemi distribuiti.

3.2 Analisi Dettagliata dei Nodi Subscriber

I nodi subscriber sono responsabili della ricezione dei comandi dal publisher e della loro traduzione in azioni fisiche. L'analisi si concentra sul nodo `pneumaticbox_traj_handler`, che gestisce il controllo dei pad pneumatici.

3.2.1 Nodo `pneumaticbox_traj_handler`: Logica e Controllo

Questo nodo, scritto in Python, agisce come il cervello del sistema pneumatico. Si sottoscrive al topic `/pressure_goal` e implementa la logica per raggiungere i setpoint di pressione desiderati per ciascuno dei 6 canali.

Gestione Asincrona dei Comandi

Data la natura fisica del sistema, un comando di inflazione o deflazione non è istantaneo. Per gestire le richieste in modo ordinato ed evitare conflitti, il nodo impiega una coda di comandi (`deque`). Quando un nuovo messaggio `goals.msg` arriva, il `goal_callback` non esegue immediatamente l'azione, ma accoda comandi separati per ogni canale. Questo garantisce che ogni obiettivo di pressione per ciascun attuatore sia gestito individualmente.

```
1 def goal_callback(self, msg):  
2     """Handle incoming pressure goals - add to queue"""  
3     pressure_goals = msg.pressure_goal
```

```

4
5     # Add a separate command to the queue for each channel
6     for channel_id in range(6):
7         pressure_goal = pressure_goals[channel_id]
8         self.command_queue.append((channel_id, pressure_goal))
9
10    rospy.loginfo("Queued %d commands. Current queue size: %d",
11                  len(pressure_goals), len(self.command_queue))

```

Listing 3.5: Callback che popola la coda di comandi

Un `rospy.Timer` esegue periodicamente il metodo `process_command_queue`, che preleva un comando dalla coda solo se il sistema non è già occupato a eseguire un'operazione precedente. Questo approccio, gestito tramite una variabile "lock" (`self.command_in_progress`), garantisce un'esecuzione sequenziale e controllata, prevenendo che nuovi comandi vengano elaborati prima che il precedente sia completato o sia scaduto un timeout di sicurezza.

```

1 def process_command_queue(self, event=None):
2     """Process the next command in the queue if not busy"""
3     if self.command_in_progress:
4         # Check if we've exceeded the timeout
5         current_time = time.time()
6         if current_time - self.command_lock_time > self.
command_timeout:
7             rospy.logwarn("Command timeout exceeded, proceeding
with next command")
8             self.command_in_progress = False
9     else:
10        # Still busy, will try again later
11        return
12

```

```

13     # If we have commands in the queue and we're not busy, process
    the next command
14     if self.command_queue and not self.command_in_progress:
15         channel_id, pressure_goal = self.command_queue.popleft()
16         self.execute_pressure_command(channel_id, pressure_goal)

```

Listing 3.6: Elaborazione sequenziale dalla coda di comandi

Algoritmo di Controllo: Bang-Bang

Per raggiungere la pressione target, il nodo non si affida a un controllo proporzionale complesso, ma implementa un controllore Bang-Bang (o on-off). Questo è un tipo di controllo a retroazione semplice ma efficace per questo tipo di applicazione.

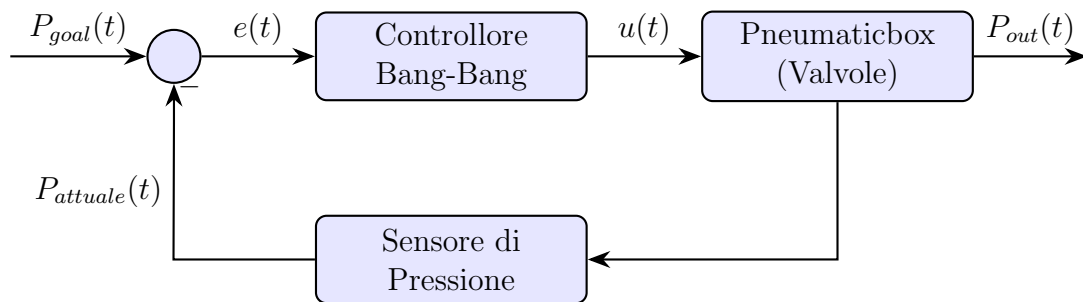


Figura 3.3: Schema a blocchi del sistema di controllo della pressione con controllore Bang-Bang.

La logica è implementata principalmente nel metodo `execute_bang_bang_control`.

1. **Calcolo dell'errore:** Viene calcolata la differenza tra la pressione target (P_{goal}) e la pressione attuale (P_{attuale}), letta dai sensori.
2. **Zona Morta (Epsilon):** Per evitare "chattering" (continue oscillazioni di accensione e spegnimento) quando la pressione è vicina al target, viene definita una banda di tolleranza, ϵ . Se l'errore ($|P_{\text{attuale}} - P_{\text{goal}}| \leq \epsilon$), il controllo si considera completato.

3. **Azione di Controllo:** Se la pressione non è nella zona di tolleranza, il controllore agisce:
- Se $(P_{\text{attuale}} < P_{\text{goal}} - \epsilon)$, invia un comando per aprire la valvola di inflazione per una durata minima e fissa (`min_inflate_duration`).
 - Se $(P_{\text{attuale}} > P_{\text{goal}} + \epsilon)$, apre la valvola di deflazione per una durata minima (`min_deflate_duration`).
4. **Ciclo Iterativo:** Dopo ogni impulso di (de)flazione, il nodo non attende passivamente, ma pianifica attivamente il controllo successivo. Utilizzando un `rospy.Timer`, richiama la funzione `check_bang_bang_progress` dopo un breve intervallo. Questo metodo legge la nuova pressione e, se necessario, richiama `execute_bang_bang_control` per applicare un altro impulso. Il ciclo continua fino a quando la pressione non rientra nella banda di tolleranza.

Questo approccio, sebbene non garantisca il raggiungimento esatto del setpoint, è computazionalmente leggero e sufficientemente preciso per l'applicazione, portando il sistema in uno stato stabile vicino alla pressione desiderata in modo robusto.

3.2.2 Misure di Sicurezza Integrate

Per garantire un funzionamento sicuro e prevenire danni all'hardware o situazioni pericolose, sono state implementate diverse misure di sicurezza.

Limite di Pressione Massima Una variabile `safety_pressure_limit` (impostata a 140 KPa) definisce una soglia critica. Durante il ciclo di controllo Bang-Bang, la pressione corrente viene costantemente monitorata. Se questo limite viene superato, il sistema interrompe immediatamente l'operazione per quel canale ed esegue una procedura di deflazione di emergenza.

```

1 # Safety check - if pressure exceeds safety limit, abort
2 if current_pressure > self.safety_pressure_limit:
3     rospy.logerr("Bang-bang: SAFETY LIMIT EXCEEDED on channel %s (%
4         s KPa). Aborting and deflating.",
5         channel_id , current_pressure)
6     # Emergency deflate
7     it_msgs = list(self.deflate(1, False, channel_id , self.delay))
8     self.airserver.submit(it_msgs)
9     self.release_command_lock()
10    return

```

Listing 3.7: Controllo del limite di sicurezza

Timeout dei Comandi Per evitare che il sistema rimanga bloccato in un ciclo infinito (ad es. a causa di un sensore difettoso o una perdita nel sistema), ogni comando ha un `command_timeout` (3 secondi). Se un comando non viene completato entro questo tempo, viene interrotto forzatamente, il "lock" viene rilasciato e il sistema procede con il comando successivo in coda.

Gestore di Spegnimento (Shutdown Handler) In caso di chiusura del nodo (ad es. tramite Ctrl+C o un comando `rostop kill`), viene attivato il metodo `shutdown_handler`. Questa funzione garantisce che tutti i canali vengano completamente sgonfiati, lasciando il sistema in uno stato sicuro.

```

1 def shutdown_handler(self):
2     """Handle shutdown properly"""
3     rospy.loginfo("Shutting down, deflating all channels")
4     it_msgs = list(self.deflate(3.0, True, 0, self.delay))
5     self.airserver.submit(it_msgs)

```

Listing 3.8: Procedura di spegnimento sicuro

Predisposizione per il Calcolo Dinamico della Durata

Sebbene l'implementazione attuale utilizzi un controllo Bang-Bang con impulsi di durata fissa, il codice include metodi per un approccio più avanzato: il calcolo dinamico della durata degli impulsi. Le funzioni `calculate_inflation_duration` e `calculate_deflation_duration` sono state sviluppate sulla base di dati sperimentali per stimare il tempo necessario per raggiungere un certo differenziale di pressione.

L'idea è di rendere l'impulso proporzionale all'errore: un errore grande genera un impulso più lungo, mentre un errore piccolo ne genera uno più breve. Questo potrebbe portare a una convergenza più rapida e a un'oscillazione minore attorno al setpoint. Tuttavia, l'implementazione di un tale controllo si scontra con significative sfide pratiche legate alla non-linearità e variabilità del sistema fisico:

- **Inconsistenza dei Componenti:** Ogni pad pneumatico può presentare una resistenza meccanica leggermente diversa, alterando il flusso d'aria e la velocità di inflazione.
- **Variazioni della Pressione di Alimentazione:** La criticità maggiore risiede nel fatto che la pressione fornita dalla `PneumaticBox` dipende direttamente da quella erogata dal compressore centrale. Quest'ultima può fluttuare durante l'operatività, specialmente se più canali vengono attivati contemporaneamente.

Di conseguenza, la relazione tra il tempo di apertura di una valvola e l'incremento di pressione risultante (il ΔP) non è costante. Ad esempio, un'apertura di 0.02 secondi può produrre variazioni di pressione molto diverse a seconda dello stato attuale del compressore.

Un controllo basato su un modello (come quello proporzionale alla durata) diventerebbe inaffidabile, rischiando di superare ampiamente il target (overshooting) o di non raggiungerlo affatto.

```

1 def calculate_inflation_duration(self, target_pressure,
   current_pressure):
2     """Calculate duration needed for inflation based on
   experimental data."""
3     pressure_diff = target_pressure - current_pressure
4     # Experimental: Safer rate based on faster balloon (0.02s for
   64 KPa) -> 0.0003125 s/KPa
5     calculated_duration = 0.0003125 * pressure_diff
6     return max(0.001, min(0.5, calculated_duration))
7
8 def calculate_deflation_duration(self, current_pressure,
   target_pressure):
9     """Calculate duration needed for deflation based on
   experimental data."""
10    pressure_diff = current_pressure - target_pressure
11    # Experimental: 0.008s for 5 KPa -> 0.008/5 = 0.0016 s/KPa
12    calculated_duration = 0.0016 * pressure_diff
13    return max(0.001, min(1.0, calculated_duration))

```

Listing 3.9: Calcolo della durata dell'impulso basato sull'errore

Questa funzionalità, sebbene presente nel codice, non è utilizzata dal controllore Bang-Bang attuale, che privilegia la robustezza di impulsi minimi e fissi. Tuttavia, rappresenta una predisposizione per future evoluzioni del sistema di controllo, che potrebbero implementare un controllore Proporzionale (P) o Proporzionale-Integrativo (PI) agendo sulla durata dell'apertura delle valvole anziché sulla loro semplice attivazione on/off.

3.2.3 Nodo `gripController`: Interfaccia di Controllo del Gripper

Parallelamente al nodo `pneumaticbox_traj_handler`, che gestisce l'attuazione pneumatica, opera il nodo `gripController`, implementato nello script `readSerial.py`. Questo nodo agisce come il secondo "attuatore" software del sistema. La sua responsabilità è quella di intercettare i comandi di alto livello per il gripper, estrarli dal messaggio ROS e trasmetterli attraverso la porta seriale. È importante notare che questo nodo non comunica direttamente con l'hardware di controllo finale del gripper (la scheda WEMOS D1 mini). Invece, la sua funzione è quella di ponte: il comando viene inviato via seriale al LILYGO T-Dongle S3. Quest'ultimo dispositivo ha poi la responsabilità di gestire la logica di comunicazione per inoltrare il comando effettivo alla WEMOS D1 mini, che a sua volta aziona l'attuatore meccanico a tendini.

Il nodo si sottoscrive allo stesso topic `/pressure_goal`, ma la sua logica interna è progettata per filtrare selettivamente i dati, ignorando completamente le informazioni sulla pressione e agendo unicamente sul campo `grip` del messaggio.

Considerazioni Architettureali: il Ponte Seriale

A prima vista, l'architettura che impiega un nodo ROS per scrivere su una porta seriale, letta poi da un dispositivo intermedio (il LILYGO Dongle) che a sua volta comunica con il controller finale, potrebbe apparire inutilmente complessa. Una soluzione più diretta sembrerebbe essere l'utilizzo di pacchetti come `rosserial`, che permettono a microcontrollori come la WEMOS D1 mini di diventare direttamente nodi ROS a tutti gli effetti, sottoscrivendosi ai topic ed eliminando la necessità di un ponte.

Tuttavia, questa scelta progettuale è una decisione pragmatica dettata dai vincoli del progetto. L'intero sistema è sviluppato su ROS 1 (Kinetic), una versione matura ma meno recente del framework. Durante lo sviluppo, si è riscontrato che alcune librerie esterne, inclusa `rosserial` per la specifica piattaforma hardware, presentavano problemi di stabilità e supporto.

Per garantire l'affidabilità e la robustezza del sistema, si è optato per una soluzione più semplice e controllata: un protocollo di comunicazione seriale personalizzato e minimale. Questo approccio, sebbene introduca un intermediario, ha il vantaggio significativo di essere completamente indipendente da librerie esterne potenzialmente non supportate, riducendo la complessità di debug e aumentando la prevedibilità del comportamento del sistema.

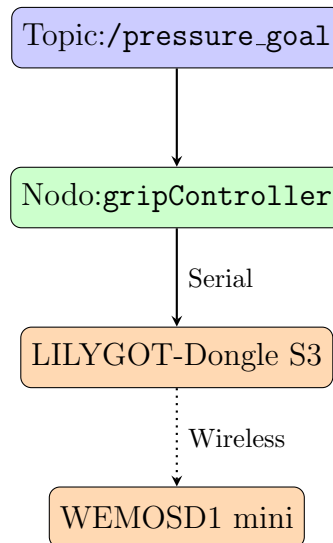


Figura 3.4: Flusso di comunicazione per il controllo del gripper, che evidenzia il ruolo di ponte del LILYGO Dongle.

Logica Operativa e Gestione della Comunicazione

Il cuore operativo del nodo risiede nella sua funzione di callback, `pressure_goal_callback`, che viene eseguita ad ogni ricezione di un messaggio dal topic `/pressure_goal`.

Filtraggio ed Estrazione del Comando

La funzione di callback, mostrata nel Listato 3.10, ha il compito di estrarre l'attributo `msg.grip` dal messaggio in arrivo. Con la struttura del messaggio ottimizzata `goals.msg`, il nodo riceve un unico messaggio per passo temporale, contenente sempre un comando valido per il gripper.

Tuttavia, è analizzando il comportamento con la precedente e inefficiente struttura `goal.msg` che emerge un problema di sincronizzazione latente. In quel contesto, il nodo riceveva 6 messaggi per passo temporale, ma solo il primo conteneva un comando `grip` utile. I cinque messaggi successivi, privi di tale campo, venivano scartati tramite la gestione dell'eccezione `AttributeError`. Questo flusso di messaggi ridondanti, sebbene inefficiente, creava involontariamente un ritardo di "cinque messaggi" tra un comando `grip` e il successivo. Questo intervallo di tempo, di fatto, agiva come un meccanismo di flow control grezzo, garantendo che il sistema di comunicazione wireless avesse tempo a sufficienza per trasmettere ogni comando senza perdite.

L'adozione del messaggio unificato `goals.msg` elimina questi messaggi vuoti, rendendo la comunicazione molto più rapida ed efficiente. Questa ottimizzazione, però, smaschera il problema di sincronizzazione precedentemente nascosto: se i comandi vengono pubblicati sul topic ROS a una frequenza troppo elevata, c'è il rischio concreto che la comunicazione seriale e poi wireless verso il gripper non riesca a gestire il flusso, portando alla perdita di comandi. Per ovviare a questa criticità, diventa indispensabile implementare una logica di gestione dei messaggi più robusta, come una coda di comandi (queue) direttamente sul microcontrollore del gripper, in modo da poter accodare i comandi in arrivo e processarli al giusto ritmo.

```
1 def pressure_goal_callback(msg):  
2     """  
3     Callback for /pressure_goal topic.
```

```
4     Extracts 'grip' and writes it to serial.
5     """
6     try:
7         grip_value = msg.grip # Estrae l'attributo 'grip'
8
9         if arduino.is_open:
10             # Invia il comando codificato sulla porta seriale
11             arduino.write(grip_value.encode())
12             rospy.loginfo(f'Sent grip value: {grip_value} to serial
13 port.')
```

Listing 3.10: Callback per l'estrazione e l'invio del comando di presa.

Una volta estratto il valore, questo viene codificato in formato byte e trasmesso attraverso la porta seriale precedentemente inizializzata.

Gestione delle Risorse

Lo script `readSerial.py` include controlli sulle risorse hardware e meccanismi di gestione degli errori nelle fasi critiche del suo ciclo di vita.

Inizializzazione Sicura della Porta Seriale

L'avvio del nodo è protetto da un blocco `try-except` che gestisce il caso critico in cui la porta seriale non sia disponibile (ad es. l'hardware non è connesso o il nome della porta è errato). Come si vede nel Listato 3.11, un fallimento nell'apertura della porta viene registrato come errore fatale tramite `rospy.logfatal` e causa l'immediata terminazione del nodo. Questo previene un funzionamento anomalo in cui il nodo rimarrebbe attivo ma incapace di eseguire la sua funzione primaria.

```
1 try:
2     arduino = serial.Serial(port='/dev/ttyACM1', baudrate=115200,
3                             timeout=.1)
4 except serial.SerialException as e:
5     rospy.logfatal("Fatal Error: Could not open serial port")
6     exit() # Termina l'esecuzione se la porta non e' accessibile
```

Listing 3.11: Gestione degli errori durante l'inizializzazione della porta seriale.

Gestore di Spegnimento (Shutdown Handler)

Per garantire che le risorse di sistema vengano rilasciate correttamente, il nodo implementa una logica di pulizia all'interno di un blocco `finally`. Questa sezione di codice viene eseguita incondizionatamente alla terminazione del nodo (sia essa normale, tramite `Ctrl+C`, o dovuta a un'eccezione).

```
1 finally:
2     if 'arduino' in globals() and arduino.is_open:
3         arduino.close()
4         rospy.loginfo('Serial port closed.')
5     else:
6         rospy.loginfo('Serial port was already closed or not
7 initialized.')
```

Listing 3.12: Chiusura sicura della porta seriale.

Come mostrato nel Listato 3.12, questo gestore verifica se l'oggetto `arduino` esiste ed è aperto, e in tal caso invoca il metodo `arduino.close()`. Questa pratica è fondamentale per rilasciare il controllo della porta seriale, prevenendo che rimanga "bloccata" e rendendola immediatamente disponibile per altri processi senza necessità di un riavvio fisico del dispositivo. Questa gestione esplicita delle risorse è analoga, per finalità, allo `shutdown_handler` implementato nel nodo `pneumaticbox_traj_handler` per garantire lo sgonfiaggio dei canali.

3.3 CoGripperDongle: Il Ponte Wireless

Il nodo `gripController`, come descritto in precedenza, non comunica direttamente con l'hardware finale del gripper, ma agisce come un'interfaccia tra il mondo ROS e una porta seriale. Il componente che riceve questi dati seriali è un microcontrollore LILYGO T-Dongle S3, il cui firmware ('CoGripperDongleFirmware.ino') svolge il ruolo critico di **ponte di comunicazione**. La sua responsabilità è duplice:

1. **Ricevere e interpretare** i comandi inviati dal nodo ROS sulla porta seriale USB.
2. **Tradurre e inoltrare** questi comandi in modo affidabile al controller del gripper (la scheda WEMOS D1 mini) attraverso un protocollo di comunicazione wireless.

3.3.1 Logica di Ricezione e Parsing Seriale

Il cuore della logica di ricezione è implementato all'interno della funzione `loop()`. A differenza di un semplice re-invio di byte, il firmware implementa un protocollo di messaggistica personalizzato per distinguere tra diversi tipi di comandi. Il mec-

canismo di framing si basa su un carattere di inizio, '\$', che segnala l'arrivo di un nuovo comando.

```
1 void loop() {
2     int bytes_available = Serial.available();
3     if (bytes_available > 0) {
4         start_byte = Serial.read();
5         if (start_byte == '$') { // Carattere di inizio comando
6             char first_byte = Serial.read();
7             if (first_byte == 'O' || first_byte == 'C' || first_byte == '
S' || first_byte == 'V') {
8                 char second_byte = Serial.read();
9                 coGripperCMD.cmd = first_byte;
10                coGripperCMD.aux = second_byte;
11                enqueueCoGripperCMD();
12                // ... scarta byte in eccesso
13            } else if (first_byte == 'N') {
14                // Comando per impostare le posizioni
15                coGripperPos.motor1_pos = Serial.parseInt();
16                coGripperPos.motor2_pos = Serial.parseInt();
17                coGripperPos.motor3_pos = Serial.parseInt();
18                enqueueCoGripperPos();
19            }
20        }
21    }
22    // ...
23 }
```

Listing 3.13: Logica di parsing dei comandi seriali nel loop().

Come mostrato nel Listato 3.13, dopo aver rilevato il carattere '\$', il firmware ispeziona il byte successivo per determinare la natura del comando. Sono gestiti due formati di messaggio principali:

- **Comandi Discreti (CoGripperCMD):** Caratteri come ‘O’ (Open), ‘C’ (Close), ‘S’ (Stop) e ‘V’ (Velocity) rappresentano azioni predefinite. Questi comandi vengono catturati insieme a un byte ausiliario (‘aux’) e memorizzati in una struttura `CoGripperCMD`. L’ausiliario permette di specificare sotto-comandi, come il motore target (‘1’, ‘2’, ‘3’) o la modalità di velocità (‘S’ per Slow, ‘F’ per Fast).
- **Comandi di Posizione Assoluta (CoGripperPos):** Il carattere ‘N’ (New Position) indica un comando per impostare direttamente la posizione angolare dei motori. In questo caso, il firmware utilizza la funzione `Serial.parseInt()` per leggere tre valori interi consecutivi dal buffer seriale, che corrispondono alle posizioni desiderate per i tre motori. Questi valori vengono memorizzati in una struttura `CoGripperPos`.

Questa logica di parsing trasforma un flusso di byte seriale in strutture dati C++ ben definite, pronte per essere trasmesse.

3.3.2 Architettura di Invio Asincrono: Coda e Task Dedicato

Un possibile problema da evitare è quello di inviare i dati in modalità wireless direttamente dalla funzione `loop()`. Questo approccio creerebbe un accoppiamento stretto tra la ricezione seriale e la trasmissione wireless, con il rischio di bloccare la lettura di nuovi comandi in attesa del completamento di un invio.

Per superare questa limitazione, il firmware adotta un’architettura *producer-consumer* utilizzando le funzionalità del sistema operativo FreeRTOS, che è alla base del framework Arduino per ESP32.

- La funzione `loop()` agisce da **producer**. Invece di inviare direttamente i dati, il suo unico compito è quello di "impacchettare" il comando ricevuto in una struttura `DataPacket` e inserirlo in una coda ('sendQueue') tramite le funzioni `enqueueCoGripperCMD()` o `enqueueCoGripperPos()`. Questa operazione è estremamente rapida e non bloccante.
- Un task separato, `sendTask`, agisce da **consumer**. Questo task viene eseguito in parallelo e il suo unico scopo è attendere che un pacchetto diventi disponibile nella coda. Una volta ricevuto un pacchetto, lo invia tramite ESP-NOW e si mette in attesa del successivo.

Questa architettura disaccoppiata (Listati 3.14 e 3.15) offre benefici significativi:

```

1 void enqueueCoGripperPos() {
2     DataPacket packet;
3     memcpy(packet.mac_addr, peer1.peer_addr, 6);
4     packet.data = (uint8_t *)malloc(sizeof(coGripperPos));
5     memcpy(packet.data, &coGripperPos, sizeof(coGripperPos));
6     packet.len = sizeof(coGripperPos);
7     if (xQueueSend(sendQueue, &packet, portMAX_DELAY) != pdTRUE) {
8         free(packet.data);
9     }
10 }
```

Listing 3.14: Funzione di accodamento del pacchetto `CoGripperPos`.

```

1 void sendTask(void *pvParameter) {
2     DataPacket packet;
3     for (;;) {
4         if (xQueueReceive(sendQueue, &packet, portMAX_DELAY) == pdTRUE)
5         {
6             esp_err_t result = esp_now_send(packet.mac_addr, packet.data,
7             packet.len);
8         }
9     }
10 }
```

```

6      // ... gestione risultato
7      free(packet.data);
8  }
9      vTaskDelay(pdMS_TO_TICKS(67)); // Limita la frequenza a circa
15 Hz
10 }
11 }

```

Listing 3.15: Task dedicato all’invio dei dati dalla coda tramite ESP-NOW.

- **Responsività:** Il `loop()` rimane sempre libero per leggere nuovi dati dalla porta seriale, prevenendo la perdita di comandi anche se la comunicazione wireless fosse temporaneamente lenta o inaffidabile.
- **Buffering:** La coda (`sendQueue`) funge da buffer elastico, capace di assorbire raffiche di comandi inviati rapidamente dal nodo ROS.
- **Controllo di Flusso (Flow Control):** Il `vTaskDelay` all’interno del `sendTask` introduce un ritardo deliberato, limitando la frequenza di invio a circa 15 Hz. Questo è un meccanismo di flow control esplicito e fondamentale. Previene di saturare il canale wireless e il microcontrollore ricevente, un problema di sincronizzazione che, come discusso nel paragrafo 3.3, era mascherato dall’inefficienza del precedente messaggio `goal.msg`.

3.3.3 Protocollo di Comunicazione Wireless: ESP-NOW

La comunicazione tra il dongle e la scheda WEMOS D1 mini del gripper è affidata al protocollo **ESP-NOW** di Espressif. Questa è una scelta progettuale pragmatica per diverse ragioni:

- **Leggerezza:** ESP-NOW è un protocollo connectionless che non richiede l'overhead di un handshake di connessione Wi-Fi completo (come TCP/IP), rendendolo ideale per inviare piccoli pacchetti di dati a bassa latenza.
- **Semplicità:** L'accoppiamento tra i dispositivi si basa semplicemente sulla conoscenza reciproca degli indirizzi MAC, configurati staticamente nel firmware.
- **Affidabilità:** Pur essendo basato su UDP, il protocollo include a livello di link dei meccanismi di acknowledgement per confermare la ricezione dei pacchetti.

Nel file `CoGripperDongleFirmware.ino`, la configurazione avviene nella funzione `setup()`, dove il dispositivo viene inizializzato e l'indirizzo MAC del gripper (`mac_peer1`) viene registrato come "peer". L'invio dei dati, come visto nel `sendTask`, avviene tramite una singola chiamata alla funzione `esp_now_send`. I dati trasmessi sono una copia binaria diretta delle strutture C++ (`CoGripperCMD` o `CoGripperPos`). Questo approccio è estremamente efficiente dal punto di vista della larghezza di banda, ma richiede che il firmware ricevente (`haria_cogripper.ino`) sia compilato con definizioni identiche di tali strutture per garantire una corretta deserializzazione.

In conclusione, il firmware del dongle svolge il ruolo di nodo di elaborazione intermedio, implementando un protocollo di messaggistica, una trasmissione asincrona e un controllo di flusso esplicito per favorire una comunicazione affidabile e disaccoppiata tra il sistema di controllo di alto livello e l'attuatore finale.

3.4 Firmware del Gripper: Ricezione ed Esecuzione Comandi

L'ultimo anello della catena di controllo è il firmware `haria_cogripper.ino`, eseguito sulla scheda WEMOS D1 mini, che costituisce il cervello dell'attuatore fina-

le. Questo componente è responsabile della ricezione dei comandi wireless, della loro interpretazione e della traduzione in movimenti fisici dei servomotori Dynamixel. Per garantire robustezza e prevenire la perdita di comandi, l'architettura software è stata riprogettata attorno a un modello *producer-consumer*, disaccoppiando completamente la ricezione dei dati dalla loro elaborazione.

3.4.1 Architettura Asincrona con Coda di Comandi (Producer-Consumer)

La comunicazione tra il *CoGripperDongle* e il gripper avviene tramite il protocollo ESP-NOW. La criticità principale in un sistema event-driven è la gestione di comandi che arrivano a una frequenza superiore a quella con cui possono essere processati. Un approccio basato su un semplice flag (come `service_CMD`) è suscettibile a race condition, dove un nuovo comando può sovrascrivere il precedente prima che venga servito.

Per risolvere questa problematica, è stata implementata una coda di comandi utilizzando le funzionalità del sistema operativo real-time FreeRTOS, integrato nel framework Arduino per ESP32.

1. **Producer:** La funzione di callback di ESP-NOW, `onDataReceiver`, assume il ruolo di produttore. Il suo unico compito è ricevere i byte in arrivo, ricostruire la struttura del comando (`CoGripperCMD`) e inserirla immediatamente in una coda a dimensione fissa (`commandQueue`). Questa operazione, eseguita tramite `xQueueSendFromISR`, è ottimizzata per essere chiamata da una Interrupt Service Routine (ISR), è non-bloccante e richiede un tempo di esecuzione minimo. Ciò garantisce che il sistema sia immediatamente pronto a ricevere il pacchetto successivo, anche se arrivasse a pochi millisecondi di distanza.

2. **Consumer:** Il `loop()` principale del firmware agisce da consumatore. A ogni ciclo, interroga la coda in modo non-bloccante tramite `xQueueReceive`. Se un comando è presente, viene estratto dalla coda e passato a una funzione dedicata, `processCMD`, per l'elaborazione.

Questo design disaccoppia la logica di controllo (lenta e complessa) dalla routine di ricezione dati (che deve essere il più veloce possibile), garantendo che nessun comando venga perso finché c'è spazio nella coda.

Implementazione del Sistema a Coda

L'inizializzazione della coda avviene nella funzione `setup()`, dove viene allocata la memoria necessaria per contenere un numero predefinito di comandi.

```

1 // Handle per la coda di comandi
2 QueueHandle_t commandQueue;
3 const int COMMAND_QUEUE_LENGTH = 10;
4
5 void setup() {
6     // ... altre inizializzazioni ...
7
8     // Crea una coda per contenere fino a 10 comandi
9     commandQueue = xQueueCreate(COMMAND_QUEUE_LENGTH, sizeof(
        CoGripperCMD));
10
11     if (commandQueue == NULL) {
12         Serial.println("Errore nella creazione della coda!");
13         while(1); // Blocco del sistema
14     }
15
16     // ... inizializzazione WiFi, ESP-NOW, etc. ...

```

```
17 }
```

Listing 3.16: Creazione della coda di comandi in `setup()`.

La funzione di callback `onDataReceiver` viene modificata per inserire i dati ricevuti direttamente nella coda, eliminando la necessità del flag `service_CMD`.

```
1 void onDataReceiver(const esp_now_recv_info_t *recv_info,
2                     const uint8_t *incomingData, int len) {
3     if (len == sizeof(CoGripperCMD)) {
4         CoGripperCMD receivedCmd;
5         memcpy(&receivedCmd, incomingData, sizeof(receivedCmd));
6
7         // Invia il comando alla coda dalla ISR. Non bloccare se piena.
8         xQueueSendFromISR(commandQueue, &receivedCmd, NULL);
9     }
10    // ... gestione altri tipi di messaggio ...
11 }
```

Listing 3.17: Callback `onDataReceiver` come "Producer".

Infine, il `loop()` principale estrae e processa i comandi in modo sicuro.

```
1 void loop() {
2     // Gestione input locale (es. pulsanti)
3     ButtonState currentState = buttonController.getButtonStates(false
4     );
5     controller.handleButtonStates(currentState);
6
7     // Controlla se ci sono comandi nella coda
8     CoGripperCMD receivedCmd;
9     if (xQueueReceive(commandQueue, &receivedCmd, 0) == pdPASS) {
10        // Se un comando e' stato ricevuto, processalo
11        processCMD(receivedCmd);
12    }
```

```
13 // Esegui il ciclo di controllo principale dei motori
14 controller.controlLoop();
15 delay(2);
16 }
```

Listing 3.18: Loop principale come "Consumer".

3.4.2 Elaborazione dei Comandi e Controllo Motori

Una volta che un comando viene estratto dalla coda, la funzione `processCMD` lo interpreta e invoca i metodi appropriati sull'oggetto `controller`, che a sua volta incapsula la logica di controllo per i singoli motori.

```
1 void processCMD(const CoGripperCMD& cmdToProcess) {
2     if (cmdToProcess.cmd == 'C') { // CLOSE
3         if (cmdToProcess.aux == '1') {
4             controller.motor1.setGoalPosition(controller.motor1.
getEndPositionMin());
5         } else if (cmdToProcess.aux == '2') {
6             // ...
7         } else {
8             controller.closeAllMotors();
9         }
10    }
11
12    if (cmdToProcess.cmd == 'O') { // OPEN
13        // ... logica simile per l'apertura ...
14        controller.openAllMotors();
15    }
16
17    if (cmdToProcess.cmd == 'S') { // STOP
18        controller.stopAllMotors();
19    }
```

```
20
21  if (cmdToProcess.cmd == 'V') { // VELOCITY
22      if (cmdToProcess.aux == 'S') { // SLOW
23          controller.motor1.changeMotorVelocity(LOW);
24          controller.motor2.changeMotorVelocity(LOW);
25          controller.motor3.changeMotorVelocity(LOW);
26      }
27      if (cmdToProcess.aux == 'F') { // FAST
28          // ...
29      }
30  }
31 }
```

Listing 3.19: Logica di interpretazione dei comandi.

È fondamentale notare che `processCMD` imposta solo la *posizione obiettivo* (`goalPosition`) o altri parametri di alto livello (come la velocità del profilo). Non si occupa direttamente di inviare comandi al bus Dynamixel. Questo compito è delegato al ciclo di controllo principale.

3.4.3 Il Ciclo di Controllo Continuo

Indipendentemente dalla ricezione di comandi esterni, la funzione `controller.controlLoop()` viene eseguita a ogni iterazione del `loop()` principale. Questa funzione rappresenta il cuore del controllo a basso livello, un ciclo reattivo e continuo che garantisce l'autonomia e la sicurezza del gripper. Le sue responsabilità sono orchestrate in una sequenza precisa per ogni motore.

Acquisizione dello Stato (Sensing) Il primo passo è la raccolta di dati telemetrici da ciascun motore Dynamixel. Le funzioni `motor.updatePosition()` e `motor.updateLoad()` leggono i registri interni dei servo, rispettivamente `Present`

Position (indirizzo 132) e **Present Load** (indirizzo 126). Questi valori forniscono un'istantanea precisa della posizione angolare attuale e del carico meccanico (proporzionale alla corrente assorbita) a cui il motore è sottoposto.

Aggiornamento della Macchina a Stati Con i dati aggiornati, la funzione `motor.updateStatus()` classifica il comportamento del motore in uno stato definito. Interroga il registro **Moving** (indirizzo 122) del motore:

- Se il motore si sta muovendo, lo stato diventa **OPENING** o **CLOSING** a seconda della direzione del movimento rispetto alla posizione obiettivo.
- Se il motore è fermo, lo stato viene determinato dalla posizione e dal carico. Se la posizione è vicina al limite massimo, lo stato è **OPEN**. Se il carico è elevato, indica che il motore sta stringendo un oggetto e lo stato diventa **GRASP**. Se il motore è fermo in una posizione intermedia con un carico basso, lo stato è **HALF_CLOSED**.

Questa macchina a stati è essenziale per la logica di controllo successiva.

Logica Adattiva di Presa (Anti-Stallo) Questa è la fase più critica del controllo autonomo, gestita da `motor.handleTorqueLimitStatus_new()`. Questo algoritmo previene il surriscaldamento del motore e garantisce una presa stabile e adattiva.

- **Rilevamento Stallo:** Se il carico supera una soglia elevata (es. 400 su 1023) e il motore non riesce a raggiungere la sua posizione obiettivo, il sistema rileva una condizione di stallo.
- **Azione Correttiva:** Invece di continuare a forzare, il controllore entra in una modalità di "presa secondaria" (`secondary_goal = true`). La posizione obiettivo viene leggermente arretrata per ridurre la pressione immediata.

- **Controllo di Coppia Emulato:** In questa modalità, il sistema cerca di mantenere il carico del motore all'interno di una banda ottimale (es. tra 340 e 360). Se il carico scende, la posizione obiettivo viene leggermente diminuita (stringendo di più); se sale, viene aumentata (allentando leggermente). Questo emula un controllo di coppia, mantenendo una forza di presa costante sull'oggetto senza raggiungere i limiti di corrente del motore.

Questo comportamento intelligente permette al gripper di conformarsi alla forma di un oggetto e di mantenere una presa sicura anche se l'oggetto si muove o si deforma.

Attuazione Come passo finale del ciclo, la funzione `motor.goToGoalPosition()` invia il valore finale di `goalPosition` (che potrebbe essere stato modificato dalla logica anti-stallo) al registro `Goal Position` (indirizzo 116) del motore. Questo chiude il ciclo, traducendo tutte le decisioni di alto livello in un'azione fisica.

Capitolo 4

Validazione sperimentale

4.1 Setup

L'ambiente di test per la validazione del sistema è composto dai componenti hardware e software descritti nei capitoli precedenti. L'unità centrale è il gripper ibrido, con la sua struttura a tendini azionata da servomotori Dynamixel e i pad pneumatici integrati. Il controllo della pressione dei pad è affidato alla **PneumaticBox**, un'unità a sei canali che gestisce l'infrazione e la deflazione tramite un compressore esterno. L'intera architettura è orchestrata da un personal computer su cui è in esecuzione **ROS 1 (Kinetic)**. Gli esperimenti di presa sono stati condotti utilizzando oggetti di prova, come una palla blu deformabile, visibile nelle Figure 4.8 e 4.9, per valutare l'adattabilità del gripper.

La raccolta dei dati è stata eseguita monitorando i topic ROS, in particolare il topic `/pressure_goal` su cui transitano i comandi di pressione e presa, e visualizzando i dati di pressione in tempo reale tramite uno script di monitoraggio personalizzato, `monitor_pressures.py`. Per garantire la ripetibilità e semplificare l'avvio dell'intero sistema, è stato creato uno script di shell, `ros_setup.sh`. Questo script automatizza l'avvio di tutti i componenti software necessari — `roscore`, i tre nodi ROS principali

(`pneumaticbox_traj_playback`, `pneumaticbox_traj_handler`, `gripController`) e lo script di monitoraggio — all'interno di una sessione `tmux`, consentendo un setup rapido e controllato dell'ambiente sperimentale.

4.2 Validazione del Controllore Pneumatico Bang-Bang

Questa sezione ha lo scopo di validare sperimentalmente l'efficacia del controllore Bang-Bang, implementato nel nodo `pneumaticbox_traj_handler`. L'obiettivo è dimostrare che l'algoritmo di controllo a retroazione, sebbene semplice, è in grado di portare e mantenere la pressione dei pad pneumatici a un valore di setpoint desiderato, gestendo le non-linearità del sistema fisico.

4.2.1 Controllo a Setpoint Standard

Per la prima validazione, al sistema è stato comandato di raggiungere e mantenere un setpoint di pressione costante su più canali contemporaneamente. La Figura 4.1 mostra il comportamento del controllore Bang-Bang quando deve portare e mantenere più canali a un setpoint di 60 kPa e in seguito riportarli a 0 kPa, come definito nel file di traiettoria.

L'andamento delle pressioni nei canali evidenzia chiaramente il funzionamento del controllore: si osservano impulsi di inflazione iniziali (picchi verso l'alto) per raggiungere rapidamente il target. Successivamente, il controllore emette impulsi più brevi e sporadici per contrastare la deflazione naturale del sistema (dovuta a piccole perdite o alla permeabilità del materiale), mantenendo la pressione attorno al valore desiderato.

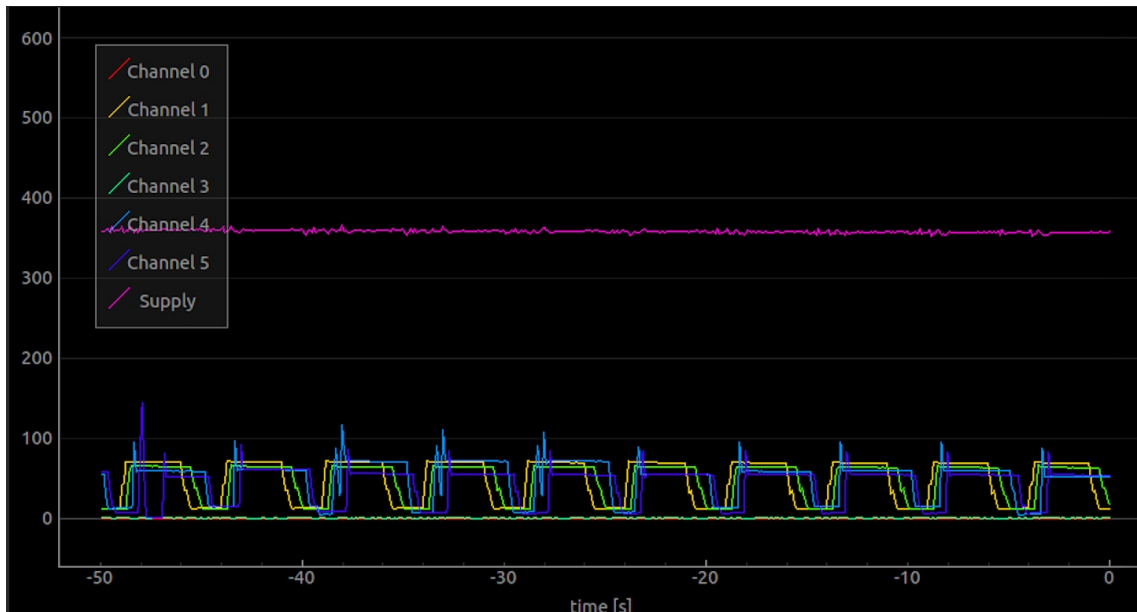


Figura 4.1: Il grafico mostra la pressione di alimentazione (Supply), e mostra la risposta in pressione dei sei canali pneumatici (Channel 0-5) a un setpoint di 60 kPa ed un successivo a 0 kPa. Si notano gli impulsi di controllo caratteristici del controllore Bang-Bang.

L'esperimento dimostra che il controllore è in grado di raggiungere e mantenere una pressione target con successo. Le oscillazioni contenute attorno al setpoint confermano l'efficacia dell'approccio a ciclo chiuso e della "zona morta" implementata per prevenire il chattering. Questo valida la robustezza e l'adeguatezza del controllore Bang-Bang per questa applicazione, che non richiede una precisione estrema ma privilegia la stabilità e la leggerezza computazionale.

È importante analizzare il comportamento del sistema anche in condizioni non ideali, come la presenza di una perdita in uno dei "polpastrelli" pneumatici. Se un pad presenta una perdita, la sua pressione interna tenderà a diminuire costantemente. Il controllore a ciclo chiuso è progettato per contrastare questo fenomeno. Come illustrato in Figura 4.2, non appena la pressione del canale (linea verde) scende al di sotto della soglia inferiore della banda di tolleranza (la "zona morta" ϵ), il controllore attiva un impulso di inflazione per riportarla all'interno della fascia desiderata.

Questo si manifesta con oscillazioni più frequenti rispetto a un canale senza perdite. La linea rossa, a confronto, mostra il decadimento più lento dovuto ad una perdita meno accentuata. Questa capacità di compensazione attiva è cruciale, poiché garantisce che il gripper possa mantenere una presa stabile e affidabile anche in caso di danneggiamento o usura di uno dei suoi componenti soft.

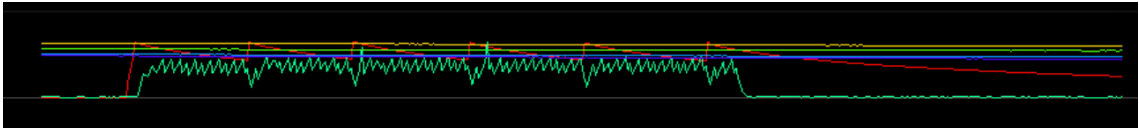


Figura 4.2: Comportamento del controllore Bang-Bang in presenza di una perdita. La linea verde mostra un canale con una perdita sotto controllo attivo, mantenuto attorno al setpoint (linea blu). La linea rossa mostra il decadimento della pressione più lento.

4.2.2 Controllo traiettoria gripper

Oltre al controllo della pressione, un obiettivo chiave della validazione è stato testare la capacità del sistema di eseguire traiettorie che coinvolgono i singoli attuatori a tendine. Questo esperimento mira a verificare la corretta interpretazione ed esecuzione di una sequenza di comandi di apertura e chiusura inviati a dita specifiche del gripper, dimostrando un controllo selettivo e non-uniforme.

È stata definita una traiettoria specifica in un file di testo, letta e pubblicata dal nodo `pneumaticbox_traj_playback`. La sequenza di comandi impartita era la seguente:

- **C1**: Comando di chiusura per il motore che aziona il dito 1.
- **O1**: Comando di apertura per il motore che aziona il dito 1.
- **C3**: Comando di chiusura per il motore che aziona il dito 3.
- **O3**: Comando di apertura per il motore che aziona il dito 3.

Questi comandi vengono pubblicati sul topic `/pressure_goal` attraverso il campo `grip` del messaggio `goals.msg`. Vengono poi intercettati selettivamente dal nodo `gripController`, che li inoltra all'hardware di controllo finale del gripper (la scheda WEMOS D1 mini) attraverso il ponte di comunicazione wireless, come descritto nel capitolo 3.

L'esecuzione dell'esperimento ha confermato che il sistema è in grado di seguire la traiettoria specificata in modo preciso e affidabile. Il dito 1 si è chiuso e successivamente riaperto al comando, seguito dalla corretta esecuzione della medesima sequenza da parte del dito 3, mentre le altre dita rimanevano immobili. Le Figure 4.3 e 4.4 illustrano qualitativamente gli stati di apertura e chiusura di un singolo dito, rappresentativi delle azioni eseguite durante il test.

L'esperimento dimostra l'efficacia dell'intera catena di comunicazione e controllo per la parte meccanica del gripper. Il sistema è in grado di gestire e attuare comandi indirizzati a sottosistemi specifici (i singoli motori), consentendo l'esecuzione di sequenze di movimento complesse. Questo risultato è fondamentale, poiché valida l'architettura software e apre alla possibilità di implementare strategie di presa avanzate, in cui il gripper può configurare la sua postura in modo asimmetrico per manipolare oggetti dalla geometria non banale.

Nota sull'utilizzo come sensore di contatto passivo: è importante sottolineare una capacità sensoriale emergente del sistema pneumatico. Oltre alla loro funzione primaria di attuazione per una presa adattiva e cedevole, i pad pneumatici possono agire come **sensori di contatto passivi**. Quando un dito, durante il suo movimento di chiusura, entra in contatto con un oggetto, la superficie del pad viene compressa. Questa deformazione ne riduce il volume interno; per la legge dei gas ideali ($PV = nRT$), una riduzione del volume (V) a parità di quantità di gas (n) e temperatura (T) provoca un aumento misurabile della pressione (P) [10]. Questo picco di pressione, rilevato istantaneamente dai sensori integrati in ogni canale della

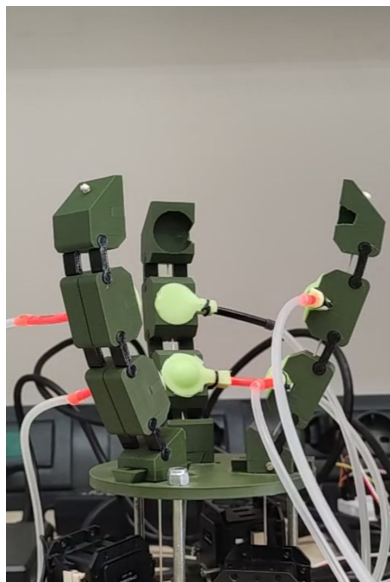


Figura 4.3: Stato dito 1 in posizione di apertura (comando 'O').

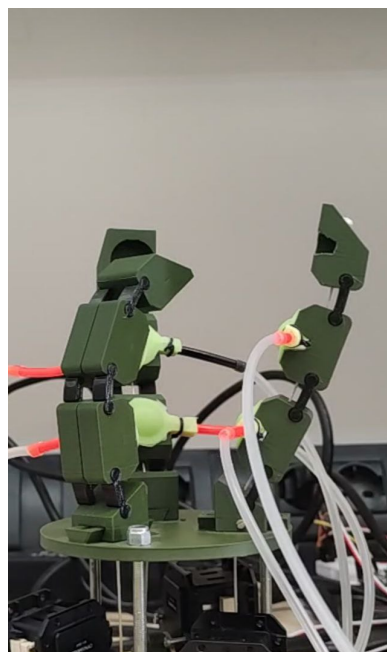


Figura 4.4: Stato dito 1 in posizione di chiusura (comando 'C').

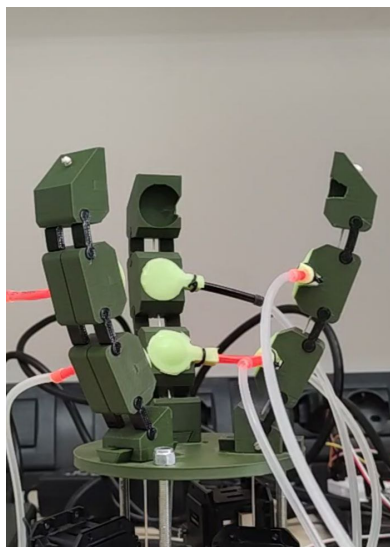


Figura 4.5: Stato dito 3 in posizione di chiusura (comando 'C').

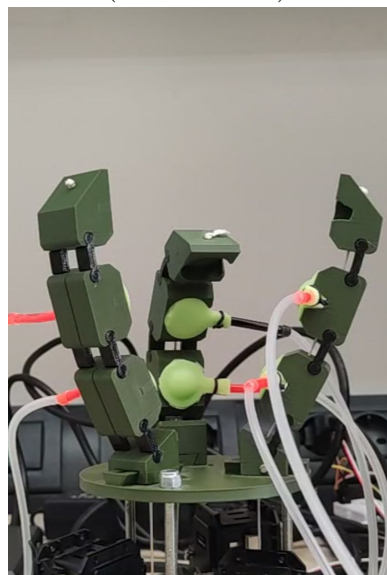


Figura 4.6: Stato dito 3 in posizione di chiusura (comando 'C').

Figura 4.7: Stati rappresentativi di un dito durante l'esecuzione della traiettoria comandata.

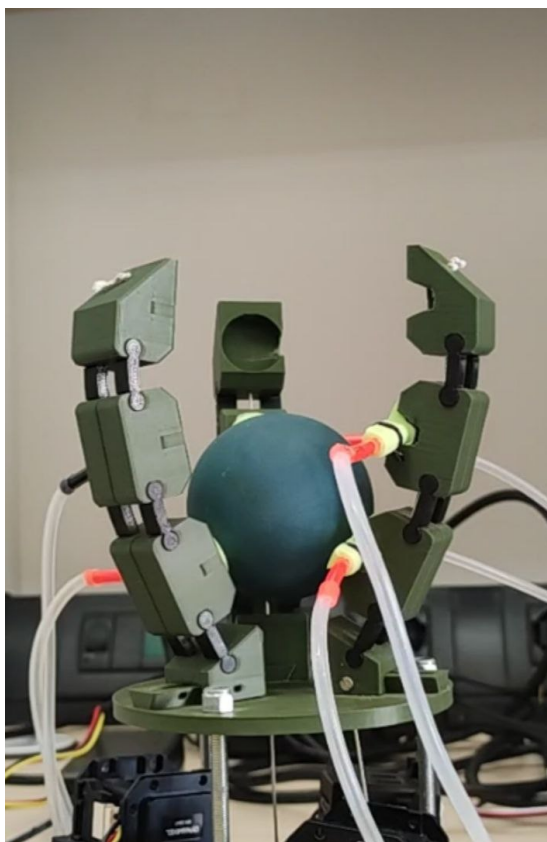


Figura 4.8: Stato di un dito in posizione di apertura (comando 'O').



Figura 4.9: Stato di un dito in posizione di chiusura (comando 'C').

Figura 4.10: Stati rappresentativi di un dito durante l'esecuzione della traiettoria comandata.

PneumaticBox, funge da segnale di contatto binario, indicando che il dito ha toccato una superficie.

La Figura 4.11 illustra questo fenomeno. Il grafico mostra la pressione del Canale 4 (linea blu), che subisce un improvviso picco nel momento in cui il polpastrello corrispondente tocca un oggetto. Gli altri canali, che non sono a contatto, mantengono una pressione nulla.

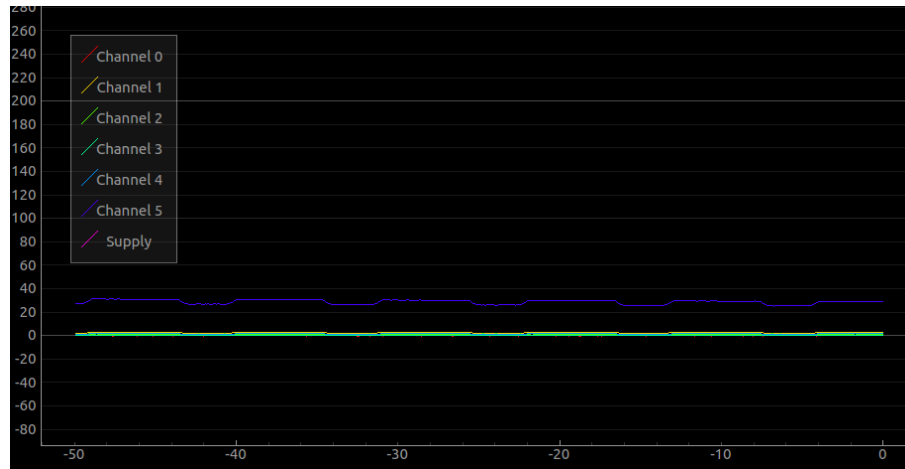


Figura 4.11: Grafico che mostra la variazione di pressione in un canale pneumatico (Channel 4, linea blu) al momento del contatto con un oggetto. Il picco di pressione agisce come un segnale di contatto binario.

4.2.3 Manipolazione In-Hand: Rotazione di una Sfera

L'esperimento finale di validazione si spinge oltre la semplice presa statica, esplorando le capacità di manipolazione *in-hand* del gripper. L'obiettivo di questa prova era dimostrare la possibilità di indurre una rotazione controllata di un oggetto trattenuto, utilizzando esclusivamente l'attuazione pneumatica, senza alcun intervento umano o movimento dei tendini del gripper.

Per questa validazione, una sfera è stata posizionata all'interno del gripper. Successivamente, è stata eseguita una routine pneumatica predefinita, caricata tramite un file di traiettoria analogo a quelli usati negli esperimenti precedenti. La sequenza di inflazione e deflazione dei sei pad pneumatici non è stata derivata da un modello analitico, ma è il risultato di un processo euristico di prove ed errori. Sono state testate numerose combinazioni fino a identificare una sequenza in grado di impartire una rotazione stabile e ripetibile all'oggetto. Tale sequenza è riportata nel file `rotazione.txt` [11].

Per documentare e visualizzare chiaramente il risultato, sono state scattate delle fotografie durante le fasi salienti della manipolazione. Come visibile nelle Figure

4.12, 4.13 e 4.14, è stato applicato un filtro alle immagini per evidenziare un segno di riferimento sulla superficie della sfera. Questa elaborazione rende inequivocabile la rotazione avvenuta.



Figura 4.12: Posizione iniziale della sfera.

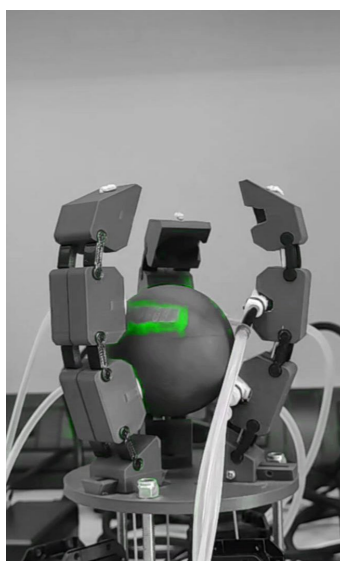


Figura 4.13: Fase intermedia della rotazione.



Figura 4.14: Posizione finale post-rotazione.

Figura 4.15: Sequenza di rotazione della sfera indotta dalla sola attuazione pneumatica. Un filtro visivo evidenzia in verde un marcatore sulla sfera per rendere apprezzabile la rotazione.

L'esperimento ha avuto successo. La sequenza pneumatica è stata in grado di modificare l'orientamento della sfera all'interno della presa, dimostrando una capacità di manipolazione fine. Questo risultato è significativo perché valida il potenziale del sistema di attuazione ibrido non solo per afferrare oggetti in modo sicuro e adattivo, ma anche per manipolarli attivamente, aprendo a scenari applicativi più complessi che richiedono destrezza e ri-orientamento dell'oggetto.

Capitolo 5

Conclusioni e Sviluppi Futuri

5.1 Conclusioni

Questo lavoro ha presentato la progettazione, l'implementazione e la validazione di un gripper robotico ibrido, capace di combinare un'attuazione a tendini per la presa di forza con un sistema di pad pneumatici per la manipolazione fine e adattiva. L'architettura di controllo, basata su ROS, ha dimostrato di orchestrare con successo i due sottosistemi eterogenei.

Gli esperimenti hanno validato l'efficacia del controllore pneumatico Bang-Bang, che si è rivelato robusto nel mantenere le pressioni di setpoint e nel compensare attivamente le perdite. Inoltre, è stata confermata la capacità del sistema di eseguire comandi selettivi sui singoli attuatori, consentendo prese asimmetriche e complesse.

Il risultato più significativo è stata la dimostrazione della capacità di manipolazione *in-hand*. Sfruttando unicamente l'attuazione pneumatica, il gripper è stato in grado di indurre una rotazione controllata su un oggetto, passando da una semplice presa statica a una manipolazione dinamica. Questo convalida il potenziale del sistema ibrido per applicazioni che richiedono un elevato grado di destrezza, andando oltre il semplice afferrare e rilasciare.

5.2 Sviluppi Futuri: Reinforcement Learning per la Manipolazione Autonoma

L'esperimento di rotazione della sfera ha validato una capacità fondamentale, ma la sequenza di attuazione è stata ottenuta attraverso un approccio iterativo di tentativi e correzioni. Questo approccio, sebbene efficace per un task specifico, non è scalabile né adattivo a oggetti o manipolazioni differenti.

Uno sviluppo futuro di grande interesse consiste nell'introdurre paradigmi di **Reinforcement Learning (RL)**. L'obiettivo sarebbe quello di permettere al sistema di *imparare* autonomamente le strategie di manipolazione ottimali.

In un simile scenario:

- **L'agente** sarebbe il sistema di controllo del gripper.
- **Le azioni** sarebbero le sequenze di inflazione e deflazione dei sei pad pneumatici.
- **Lo stato** sarebbe una combinazione dei dati sensoriali: la pressione letta da ogni pad e, idealmente, l'orientamento dell'oggetto ottenuto da un sistema di visione esterno.
- **La ricompensa (reward)** verrebbe assegnata in base al successo della manipolazione. Ad esempio, una ricompensa positiva per ogni passo che avvicina l'oggetto all'orientamento desiderato e una ricompensa negativa elevata in caso di caduta dell'oggetto o allontanamento dalla configurazione target, in modo da guidare il sistema esclusivamente verso la manipolazione corretta ed evitare traiettorie errate..

Attraverso un processo di esplorazione, l'agente RL potrebbe scoprire strategie di manipolazione complesse e contro-intuitive, superando le limitazioni di una

programmazione manuale. Questo permetterebbe al gripper non solo di eseguire un compito predefinito, ma di adattare il suo comportamento a oggetti di forme, pesi e materiali diversi, apprendendo a gestirli senza la necessità di traiettorie definite dall'utente. Tale approccio rappresenterebbe un passo significativo verso una manipolazione robotica veramente autonoma e intelligente.

Appendice A

Dynamixel MX-28

Specifica	Valore / Descrizione
Tensione di Esercizio	12V nominale (range: 11.1V–14.8V)
Coppia di Stallo	2.5 $N \cdot m$ (a 12V)
Risoluzione	4096 passi/giro (0.088°/passo)
Comunicazione	TTL seriale half-duplex (Protocollo 1.0/2.0)
Indirizzi Control Table	
11	Operating Mode
36	PWM Limit
108	Profile Acceleration
112	Profile Velocity
116	Goal Position
126	Present Load

Tabella A.1: Specifiche Chiave e Parametri di Controllo dell'Attuatore Dynamixel MX-28

Bibliografia

- [1] *Robotics*, <https://en.wikipedia.org/wiki/Robotics>
- [2] S. Raval and B. Patel, “A Review on Grasping Principle and Robotic Grippers,” *International Journal of Engineering Development and Research*, vol. 4, no. 1, pp. 483, 2016.
- [3] S. Vanni, “DESIGN AND MANUFACTURING OF A ROBOTIC GRIPPER THAT COMBINES PNEUMATIC AND TENDON-DRIVEN ACTUATION,” Master’s thesis, Prof. G. Salvietti and Ph.D. L. Franco, Academic Year 2022-2023.
- [4] P. Khadkotkar and A. Mishra, “Short Review on Gripper System: Introduction, Types and Applications,” *International Journal of Enhanced Research in Science, Technology & Engineering*, vol. 13, pp. 44-53, Feb. 2024.
- [5] E. Pianigiani, “SoftPad Gripper: sviluppo del nodo ROS per il controllo,” Tesi di laurea, Prof. G. Salvietti, relatore, Dott.essa M. Pozzi and Dott. L. Franco, correlatori, [Università, Anno Accademico].
- [6] A. Bicchi and V. Kumar, “Robotic Grasping and Contact: A Review,” in *Springer Handbook of Robotics*, B. Siciliano and O. Khatib, Eds., 2nd ed. Cham: Springer, 2016, pp. 348-351.
- [7] D. M. Rusu, S. D. Mândru, C. M. Biriş, O. L. Petraşcu, F. Morariu, and A. Ianosi-Andreeva-Dimitrova, “Soft Robotics: A Systematic Review and Bibliometric Analysis,” *Micromachines*, vol. 14, no. 2, pp. 359, 2023, doi: 10.3390/mi14020359.
- [8] Y. Zhang and Z. Wang, “Review of robotic grippers for high-speed handling of fragile foods,” *Advanced Robotics*, pp. 1–17, 2025, doi: 10.1080/01691864.2025.2508785.
- [9] Reach Robotics, “Force and Torque sensors – why are they of interest in robotics?”, 18 April, 2023. [Online]. Available: <https://reachrobotics.com/blog/force-and-torque-ft-why-are-they-of-interest-in-robotics/>

- [10] Wikipedia contributors, “Equazione di stato dei gas perfetti”, *Wikipedia, L’enciclopedia Libera*, 13 April 2025, 16:58 UTC. [Online]. Available: https://it.wikipedia.org/wiki/Equazione_di_stato_dei_gas_perfetti.
- [11] [Abdourahmane Barro], “[BallonHand]”, [2025]. [Online]. Available: <https://github.com/AbdouBarro32/BallonHand>.